IoTEventSim / IoT Simulator

Importante: Esta versão usa os diretórios e arquivos reais do repositório. O roteiro principal trabalha com 1 dispositivo, o cenário 2x2 e 10 dispositivos por região.

Objetivo: permitir que o IoTEventSim seja instalado em ambiente de teste, execute os dispositivos CoAP, configure o Mosquitto, inicie gateways, acompanhe MQTT/mapa e aplique eventos manualmente ou por ev-man.

1. Roteiro de teste

Siga o manual nesta ordem. Não pule para os eventos antes de o primeiro GET funcionar.

1. Instalar pacotes do sistema, uv, Python 3.12 e o projeto.
2. Iniciar scenarios/device-model.json e fazer um GET com iot-client.
3. Configurar o Mosquitto em /etc/mosquitto e testar start/stop/status.
4. Entender os JSONs.
5. Executar o cenário real scenarios/2x2.
6. Criar arquivos devices separados para cada região do cenário 2x2 e iniciar GW1/GW2.
7. Abrir o mapa em web/ e confirmar as mensagens MQTT.
8. Executar event-transient.json, event-permanent.json e event-moving.json.
9. Executar scenarios/schedule-rendezvous-2x2.json com ev-man.
10. Gerar 10 dispositivos por região a partir de scenarios/device-model.json e testar run-iot-range e iot-sim-batch.

Componente	Comando principal	Função
Dispositivo	uv run iot-sim <arquivo.json>	Servidor CoAP do dispositivo.
Cliente	uv run iot-client <URI> [evento.json]	GET sem arquivo; POST quando recebe evento.json.
Gateway	uv run iot-gw -i ... -d ... -t ...	Consulta CoAP e publica o resultado no MQTT.
Event Manager	uv run ev-man <schedule.json>	Agenda POSTs CoAP usando time_ms.
Mapa	web/index.html	Recebe MQTT por WebSocket e desenha os dispositivos.

2. Instalação no Ubuntu / Debian

2.1 Pacotes do sistema

O comando service é fornecido pelo pacote init-system-helpers. Mosquitto e mosquitto-clients instalam o broker e os clientes mosquitto_sub/mosquitto_pub.

```
sudo apt update
sudo apt install -y git jq curl ca-certificates uuid-runtime bc init-
system-helpers
sudo apt install -y mosquitto mosquitto-clients
```

2.2 Instalar uv e Python 3.12 – (caso tenha o UV instalado ir para etapa 2.3

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv python install 3.12
```

Nota: O projeto traz .python-version com 3.12 e pyproject.toml exige Python >= 3.12.

2.3 Obter e instalar o projeto

Para que os caminhos deste manual sejam idênticos, use a pasta ~/iot-sim-main. Se você extraiu o ZIP enviado, apenas mova/renomeie a pasta para esse local. Se for clonar:

```
cd ~
git clone https://github.com/paulo-coelho/iot-sim.git iot-sim-main
cd ~/iot-sim-main
uv venv --python 3.12
uv pip install .
```

Atenção: O ponto final em "uv pip install ." é obrigatório. Ele significa instalar o projeto da pasta atual.

Ative o ambiente virtual com comando **source .venv/bin/activate** para isolar as dependências do projeto. Isso evita conflitos entre bibliotecas e mantém o Python global limpo. O terminal mostrará (.venv) ou (iot-sim-main) indicando que está ativo. Para sair, use deactivate. **Essa etapa é opcional**

2.4 Primeiro teste - este é o teste da instalação – (Iniciar dispositivo e enviar um GET)

Terminal 1:

```
cd ~/iot-sim-main
uv run iot-sim scenarios/device-model.json
```

Terminal 2:

```
cd ~/iot-sim-main
uv run iot-client coap://127.0.0.1:5001/device/data
```

Resultado esperado: o cliente mostra Response - Received com uuid, timestamp, status, sensor_data, battery e coordinate. Se isso funcionar, siga para o Mosquitto seção.

3. Instalação no Rocky Linux

3.1 Repositórios e pacotes

No Rocky, o pacote initscripts-service fornece o comando service. O Mosquitto é obtido via EPEL.

```
sudo yum install -y dnf-plugins-core
sudo dnf config-manager --set-enabled crb
sudo yum install -y epel-release
sudo yum makecache
sudo yum install -y git curl ca-certificates jq util-linux bc
initscripts-service
sudo yum install -y mosquitto
```

3.2 uv, Python e projeto

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv python install 3.12

cd ~
git clone https://github.com/paulo-coelho/iot-sim.git iot-sim-main
cd ~/iot-sim-main
uv venv --python 3.12
uv pip install .
```

3.3 Primeiro teste

```
# Terminal 1
cd ~/iot-sim-main
uv run iot-sim scenarios/device-model.json

# Terminal 2
cd ~/iot-sim-main
uv run iot-client coap://127.0.0.1:5001/device/data
```

3.4 Firewall quando outras máquinas acessarem

Para os exemplos de até 10 dispositivos por região, as portas CoAP usadas são 5001-5010 e 6001-6010. MQTT usa 1883/TCP e o mapa usa 9001/TCP.

```
sudo firewall-cmd --permanent --add-port=1883/tcp
sudo firewall-cmd --permanent --add-port=9001/tcp
sudo firewall-cmd --permanent --add-port=5001-5010/udp
sudo firewall-cmd --permanent --add-port=6001-6010/udp
sudo firewall-cmd --reload
sudo firewall-cmd --list-ports
```

4. Windows – WSL2 + Ubuntu

Para executar os scripts Bash do repositório sem adaptar o código, use WSL2 + Ubuntu.

```
# PowerShell como Administrador
wsl --install -d Ubuntu
```

Abra o Ubuntu do WSL e siga a Seção 2. O restante do manual usa comandos Linux.

5. Mosquitto - configuração prática em /etc

5.1 Portas usadas

Porta	Protocolo	Uso
1883/TCP	MQTT	iot-gw publica; mosquitto_sub assina.
9001/TCP	MQTT sobre WebSocket	web/app.js conecta o navegador ao broker.

5.2 Configurar o Mosquitto diretamente em /etc

Para evitar comandos longos com redirecionamentos, edite os arquivos diretamente. Quando o manual usar vi: pressione i para editar; ao terminar, pressione Esc, digite :wq e pressione Enter.

Configuração principal:

```
sudo vi /etc/mosquitto/mosquitto.conf
```

Coloque exatamente:

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous true
```

Importante: allow_anonymous true é adequado apenas para uso controlado. Não exponha esse broker diretamente à Internet.

5.3 start, stop, restart e status

Com service:

```
sudo service mosquitto start
sudo service mosquitto status
sudo service mosquitto restart
sudo service mosquitto stop
```

Com systemctl (forma nativa em sistemas systemd):

```
sudo systemctl start mosquitto
sudo systemctl status mosquitto
sudo systemctl restart mosquitto
sudo systemctl stop mosquitto
```

Para continuar os testes, deixe o broker iniciado:

```
sudo service mosquitto start
sudo service mosquitto status
```

5.4 Testar MQTT antes do gateway

Terminal A:

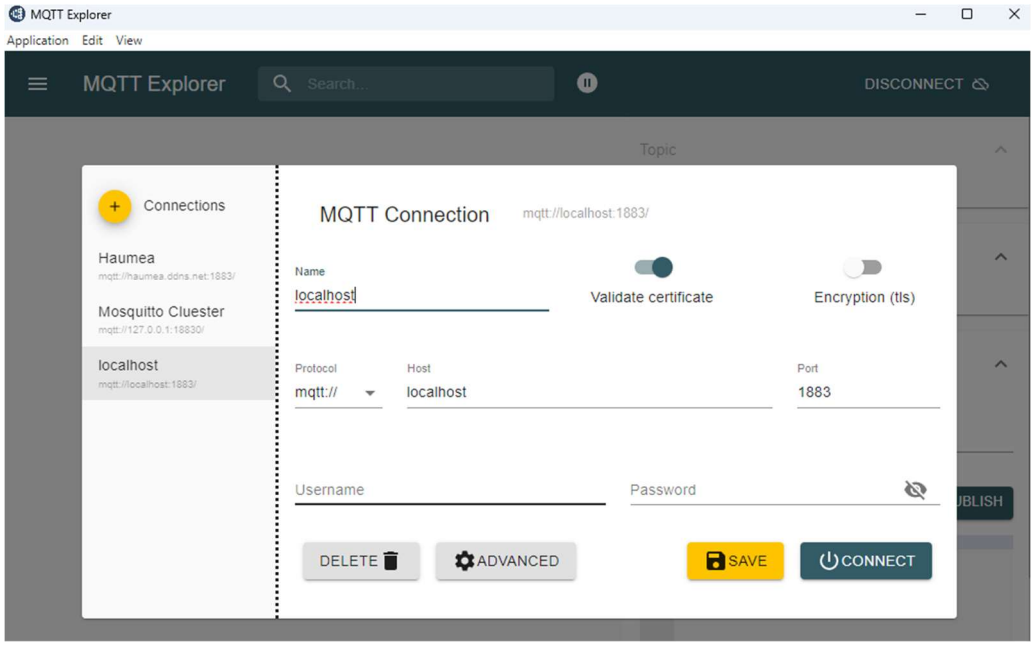
```
mosquitto_sub -h localhost -p 1883 -v -t "#"
```

Terminal B:

```
mosquitto_pub -h localhost -p 1883 -t /teste/mosquitto -m "MQTT OK"
```

O Terminal A deve mostrar: /teste/mosquitto MQTT OK.

Dica: Caso queira utilizar o MQTT-EXPLORER para visualizar as publicações em modo gráfico basta, instalar o Mqtt-Explorer pelo seguinte link: **https://mqtt-explorer.com/**



6. JSONs do IoTEventSim - o que cada campo faz

6.1 scenarios/device-model.json

```
{
  "uuid": "123e4567-e89b-12d3-a456-426614174000",
  "server_host": "127.0.0.1",
  "server_port": 5001,
  "resource_path": ["device", "data"],
  "coordinate": {"latitude": 40.7128, "longitude": -74.0060},
  "temperature_range": [20.0, 30.0],
  "pressure_range": [1000.0, 1015.0],
  "battery_charge": 100,
  "battery_transmit_discharge": 1,
  "battery_idle_discharge": 0.05,
  "drop_percentage": 5,
  "delay_profiles": [
    {"probability": 70, "min": 0.5, "max": 1.5},
    {"probability": 30, "min": 2.0, "max": 3.0}
  ]
}
```

Campo	Exemplo real	O que acontece
uuid	123e...4000	Identifica o dispositivo. Use UUID diferente para cada dispositivo.
server_host	127.0.0.1	O CoAP escuta apenas localmente. Para acesso remoto, use o IP da máquina.
server_port	5001	Porta UDP do CoAP. A URI fica coap://127.0.0.1:5001/device/data.
resource_path	["device","data"]	Forma o caminho /device/data.
coordinate	40.7128, -74.0060	Posição publicada e mostrada no mapa.
temperature_range	[20,30]	Cada GET sorteia temperatura dentro da faixa; ex.: 24.7.
pressure_range	[1000,1015]	Cada GET sorteia pressão dentro da faixa; ex.: 1007.2.
battery_charge	100	Carga inicial em porcentagem.
battery_transmit_discharge	1	Cada GET respondido/POST consome 1 ponto percentual neste exemplo.
battery_idle_discharge	0.05	A cada 60 s o dispositivo consome 0.05 ponto percentual.
drop_percentage	5	Cada GET tem 5% de chance de ser descartado.
delay_profiles	70% + 30%	O simulador escolhe automaticamente um perfil e sorteia o atraso dentro dele.

6.2 delay_profiles - como a escolha realmente funciona

Você não escolhe manualmente um dos perfis durante a execução. Em cada GET que não foi descartado, o simulador escolhe aleatoriamente um perfil usando probability como peso e depois sorteia um valor uniforme entre min e max.

```
"delay_profiles": [
  {"probability": 70, "min": 0.5, "max": 1.5},
  {"probability": 30, "min": 2.0, "max": 3.0}
]
```

Possibilidade	Interpretação
Perfil 1 - peso 70	Aproximadamente 70% das escolhas. Atraso sorteado entre 0.5 e 1.5 s.
Perfil 2 - peso 30	Aproximadamente 30% das escolhas. Atraso sorteado entre 2.0 e 3.0 s.
Exemplo de 5 GETs	Pode resultar em 0.8 s, 1.3 s, 2.7 s, 0.6 s, 1.1 s. Não existe sequência fixa.

No iot-sim e no batch, a soma de probability é validada e deve ser exatamente 100. Se quiser somente um perfil:

```
"delay_profiles": [
  {"probability": 100, "min": 1.0, "max": 2.0}
]
```

Nesse caso, 100% dos GETs que chegam à etapa de atraso usam esse único perfil, com valor sorteado entre 1 e 2 segundos.

6.3 Arquivo de lista do gateway

Exemplo real: scenarios/devices-2x2.json.

```
{
  "devices": [
    "coap://127.0.0.1:5001/device/data",
    "coap://127.0.0.1:5002/device/data",
    "coap://127.0.0.1:6001/device/data",
    "coap://127.0.0.1:6002/device/data"
  ]
}
```

Parte	Exemplo	Significado
devices	lista de URIs	O gateway percorre esses endereços continuamente.
coap://	Protocolo	O gateway faz GET usando CoAP.
127.0.0.1	Host	Máquina onde o dispositivo está rodando.
5001	Porta	Deve ser igual ao server_port do JSON do dispositivo.
/device/data	Path	Deve corresponder ao resource_path.

6.4 JSON de evento

Os três arquivos usados no tutorial são scenarios/event-transient.json, scenarios/event-permanent.json e scenarios/event-moving.json.

Campo	Exemplo no repositório	Efeito prático
event_name	Transient Event 1	Nome que passa a aparecer em status quando a transição chega ao estado alvo.
event_type	transient	Entrada -> período ativo -> retorno.
event_type	permanent	Entrada -> permanece no novo estado até outro POST ou reinício.
temperature_range	[35,45]	Move a faixa de temperatura para 35..45.
pressure_range	[998,1016]	Move a faixa de pressão para 998..1016.
battery_transmit_discharge	20	No alvo, cada transmissão custa 20 pontos percentuais.
battery_idle_discharge	4	No alvo, perde 4 pontos percentuais a cada 60 s.
drop_percentage	80	Cada GET passa a ter até 80% de chance de perda no estado alvo.
delay_profiles	100%, 5..12 s	O evento troca o delay para um único perfil.
coordinate	42.0, -76.0	Destino usado pelo event-moving.json.
transition_duration_s	20	Tempo para sair do estado atual e iniciar o estado do evento.
transient_event_duration_s	40	Somente transient: tempo em que permanece no alvo.
transient_event_return_s	30	Somente transient: tempo de retorno aos parâmetros anteriores.

Nota: battery_charge não faz parte de EventConfig. O evento pode alterar as taxas de descarga, mas não definir diretamente a carga atual.

6.5 Como ler um evento transient

No scenarios/event-transient.json: transition_duration_s=20, transient_event_duration_s=40 e transient_event_return_s=30.

Tempo aproximado	O que ocorre
t=0	POST aceito; começa a transição para o evento.
t=0..20 s	Temperatura, pressão, drop e taxas de bateria mudam gradualmente.
Evento Iniciado	delay_profiles troca para o perfil do evento.
t≈20 s	Estado alvo atingido; status passa a refletir event_name.
t≈20..60 s	Evento permanece ativo por 40 s.
t≈60..90 s	Retorno gradual para o estado anterior.
t≈90 s	Parâmetros retornáveis voltaram ao estado anterior.

Importante: Em eventos de movimento, coordinate é uma exceção: o código marca a coordenada como não transitória. Ela não volta para a posição anterior no retorno.

6.6 log-config.json - somente o necessário

O iot-sim lê log-config.json na raiz, cria a pasta logs/ e troca o nome do arquivo para logs/dev-<uuid>.log.

```
# ver logs criados
ls -lh logs/

# acompanhar um log específico
tail -f logs/dev-<UUID>.log
```

6.7 JSON do ev-man

```
[
  {
    "time_ms": 1000,
    "device": "coap://127.0.0.1:5001/device/data",
    "event": { ... }
  }
]
```

Campo	Exemplo	Significado
time_ms	1000	Dispara aproximadamente 1 s depois que o ev-man começa.
time_ms	15000	Dispara aproximadamente 15 s depois do início.
device	coap://127.0.0.1:5001/device/data	URI exata que receberá o POST.
event	{...}	Objeto EventConfig enviado ao dispositivo.

time_ms é relativo ao início do programa; não é horário do relógio nem timestamp Unix. Itens com o mesmo time_ms são programados para aproximadamente o mesmo instante.

7. Prática 1 - um dispositivo

Use primeiro o arquivo scenarios/device-model.json.

7.1 Iniciar

```
cd ~/iot-sim-main
uv run iot-sim scenarios/device-model.json
```

Ele usa 127.0.0.1:5001 e o recurso /device/data.

7.2 Consultar

```
watch -n 2 'uv run iot-client coap://127.0.0.1:5001/device/data'
```

7.3 Encerrar

No terminal do dispositivo, pressione Ctrl+C.

8. Scripts de geração e execução do projeto

Script	O que ele faz
scripts/generate-config.sh	Usa um JSON modelo, altera somente os campos informados e gera um novo UUID quando -u não é usado.
scripts/generate-config-range.sh	Repete generate-config.sh para criar vários dispositivos e também gera o arquivo devices-RR.json do gateway.
scripts/run-iot.sh	Inicia um dispositivo no formato device-RR-DDDD.json.
scripts/run-iot-range.sh	Inicia uma faixa de dispositivos, cada um em um processo Python separado.
iot-sim-batch --config-dir	Inicia todos os DeviceConfig de uma pasta dentro de um único processo Python.

8.1 Gerar um dispositivo a partir de scenarios/device-model.json

O script lê o arquivo modelo que está na pasta scenarios/device-model.json, mantém os campos que você não alterar e substitui apenas as opções passadas na linha de comando. Se -u não for informado, ele gera um UUID novo automaticamente.

A partir da raiz ~/iot-sim-main, gere um dispositivo de aula na porta 7001:

```
./scripts/generate-config.sh scenarios/device-model.json -h 127.0.0.1 -p 7001 -a -18.9185 -o -48.2595 -d 10 -c 100 > scenarios/device-aula.json
```

Inicie o dispositivo gerado:

```
uv run iot-sim scenarios/device-aula.json
```

Em outro terminal, consulte:

```
watch -n 2 'uv run iot-client coap://127.0.0.1:7001/device/data'
```

O arquivo scenarios/device-aula.json é um DeviceConfig normal. Você pode abri-lo com vi scenarios/device-aula.json para comparar com o modelo original.

8.2 Flags principais do generate-config.sh

Flag	Campo alterado	Exemplo
-h	server_host	-h 127.0.0.1
-p	server_port	-p 7001
-a	coordinate.latitude	-a -18.9185
-o	coordinate.longitude	-o -48.2595
-u	uuid	-u <UUID>
-s / -e	temperature_range	-s 18 -e 32
-m / -x	pressure_range	-m 988 -x 1000
-d	drop_percentage	-d 10
-i	battery_idle_discharge	-i 0.05
-t	battery_transmit_discharge	-t 0.5
-c	battery_charge	-c 100
-f	delay_profiles	-f '[{"probability":100,"min":0.1,"max":0.5}]'

8.3 Gerar 10 dispositivos por região a partir do mesmo modelo

Para a prática com 10 dispositivos, vamos gerar novos arquivos dentro de scenarios/2x10/. **O script generate-config-range.sh chama ./generate-config.sh internamente, por isso entre uma vez na pasta scripts/.**

```
cd ~/iot-sim-main/scripts
```

Região 1 - gera device-01-0001.json até device-01-0010.json nas portas 5001 a 5010:

```
./generate-config-range.sh ../scenarios/2x10/reg1 1 5000 10
../scenarios/device-model.json -h 127.0.0.1 -a -18.9185 -o -48.2595
```

Região 2 - gera device-02-0001.json até device-02-0010.json nas portas 6001 a 6010:

```
./generate-config-range.sh ../scenarios/2x10/reg2 2 6000 10
../scenarios/device-model.json -h 127.0.0.1 -a 48.8566 -o 2.3522
```

Cada execução também cria um devices-RR.json. Mova esses dois arquivos para scenarios/, porque iot-sim-batch tenta interpretar todo *.json do --config-dir como DeviceConfig.

```
(diretório reg1) -- mv devices-01.json ../../devices-01-10.json

(diretório reg2) -- mv devices-02.json ../../devices-02-10.json
```

Volte para a raiz do projeto:

```
cd ~/iot-sim-main
```

Importante: Se você executar novamente a geração, os arquivos de dispositivo serão sobrescritos e novos UUIDs serão criados, porque -u não foi informado.

8.4 Gerar 10 dispositivos com coordenadas próximas

Como alternativa ao generate-config-range.sh original, foi criado o script adicional generate-config-range-nearby.sh. Ele mantém os parâmetros utilizados pelo generate-config.sh, porém permite distribuir automaticamente os dispositivos ao redor de uma coordenada central Importante: este script é uma extensão experimental utilizada neste manual e está prevista para futura incorporação ao repositório oficial do IoTEventSim.

```
./generate-config-range-nearby.sh ../scenarios/2x10/reg1 1 5000 10
../scenarios/device-model.json -h 127.0.0.1 -a -18.9186 -o -48.2772 --
spacing-m 500
```

Esse parâmetro **(--spacing-m)** define a distância aproximada, em metros, entre os dispositivos gerados.

9. Prática 2 - cenário 2x2

O repositório já contém quatro dispositivos: Região 1 nas portas 5001/5002 e Região 2 nas portas 6001/6002.

Arquivo	Região	Porta
scenarios/2x2/dev11.json	1	5001
scenarios/2x2/dev12.json	1	5002
scenarios/2x2/dev21.json	2	6001
scenarios/2x2/dev22.json	2	6002

Abra quatro terminais:

```
# Terminal D1
cd ~/iot-sim-main
uv run iot-sim scenarios/2x2/dev11.json

# Terminal D2
cd ~/iot-sim-main
uv run iot-sim scenarios/2x2/dev12.json


# Terminal D3
cd ~/iot-sim-main
uv run iot-sim scenarios/2x2/dev21.json


# Terminal D4
cd ~/iot-sim-main
uv run iot-sim scenarios/2x2/dev22.json
```

Ou podemos iniciar todos os dispositivos de uma só vez pelo comando
uv run iot-sim-batch --config-dir scenarios/2x2 --log-level INFO

Teste um de cada região:

```
uv run iot-client coap://127.0.0.1:5001/device/data
uv run iot-client coap://127.0.0.1:6001/device/data
```

10. Arquivos devices e dois gateways no cenário 2x2

O gateway não recebe os DeviceConfig completos. Ele recebe um arquivo simples com uma lista de URIs CoAP no campo devices. O repositório já possui scenarios/devices-2x2.json com os quatro dispositivos; para usar um gateway por região, crie dois arquivos menores.

Crie o arquivo da Região 1:

```
vi scenarios/devices-2x2-reg1.json
```

Conteúdo:

```
{
  "devices": [
    "coap://127.0.0.1:5001/device/data",
    "coap://127.0.0.1:5002/device/data"
  ]
}
```

Crie o arquivo da Região 2:

```
vi scenarios/devices-2x2-reg2.json
```

Conteúdo:

```
{
  "devices": [
    "coap://127.0.0.1:6001/device/data",
    "coap://127.0.0.1:6002/device/data"
  ]
}
```

Com os quatro dispositivos e o Mosquitto ligados, inicie um gateway em cada terminal:

GW1 - Região 1:

```
uv run iot-gw -i 2000 -d scenarios/devices-2x2-reg1.json -b
localhost:1883 -t "/gw/001/"
```

GW2 - Região 2:

```
uv run iot-gw -i 2000 -d scenarios/devices-2x2-reg2.json -b
localhost:1883 -t "/gw/002/"
```

Argumento	Exemplo	Significado
-i	2000	Intervalo de consulta por dispositivo, em milissegundos.
-d	scenarios/devices-2x2-reg1.json	Arquivo com as URIs CoAP consultadas pelo gateway.
-b	localhost:1883	Broker MQTT. Em outra máquina, use IP/hostname:1883.
-t	/gw/001/	Tópico MQTT em que o gateway publica as respostas.

O gateway cria CSVs em logs/gw-AAAAMMDD-HHMMSS.csv.

Cenário Unificado do arquivo

```
{
  "devices": [
    "coap://127.0.0.1:5001/device/data",
    "coap://127.0.0.1:5002/device/data",
    "coap://127.0.0.1:6001/device/data",
    "coap://127.0.0.1:6002/device/data"
  ]
}
```

GW-ALL - Região 1-2:

```
uv run iot-gw -i 2000 -d scenarios/devices-2x2.json -b localhost:1883 -
t "/gw-all"
```

Visualização em terminal: mosquitto_sub -h localhost -p 1883 -v -t "/gw-all"

11. MQTT e mapa

11.1 Observar o que os gateways publicam

```
mosquitto_sub -h localhost -p 1883 -v -t "/gw/+/"
```

Você deverá ver mensagens alternando entre /gw/001/ e /gw/002/. Para depuração de todos os tópicos, use -t "#".

11.2 Abrir a página do mapa

```
cd ~/iot-sim-main
uv run python -m http.server 8000 -d web
```

No navegador, abra http://localhost:8000 e use: Broker URL = localhost; Port = 9001; Topic = /gw/+/. Depois clique Connect.

Nota: web/app.js cria a conexão WebSocket em /mqtt. O broker precisa estar com o listener 9001/protocol websockets ativo.

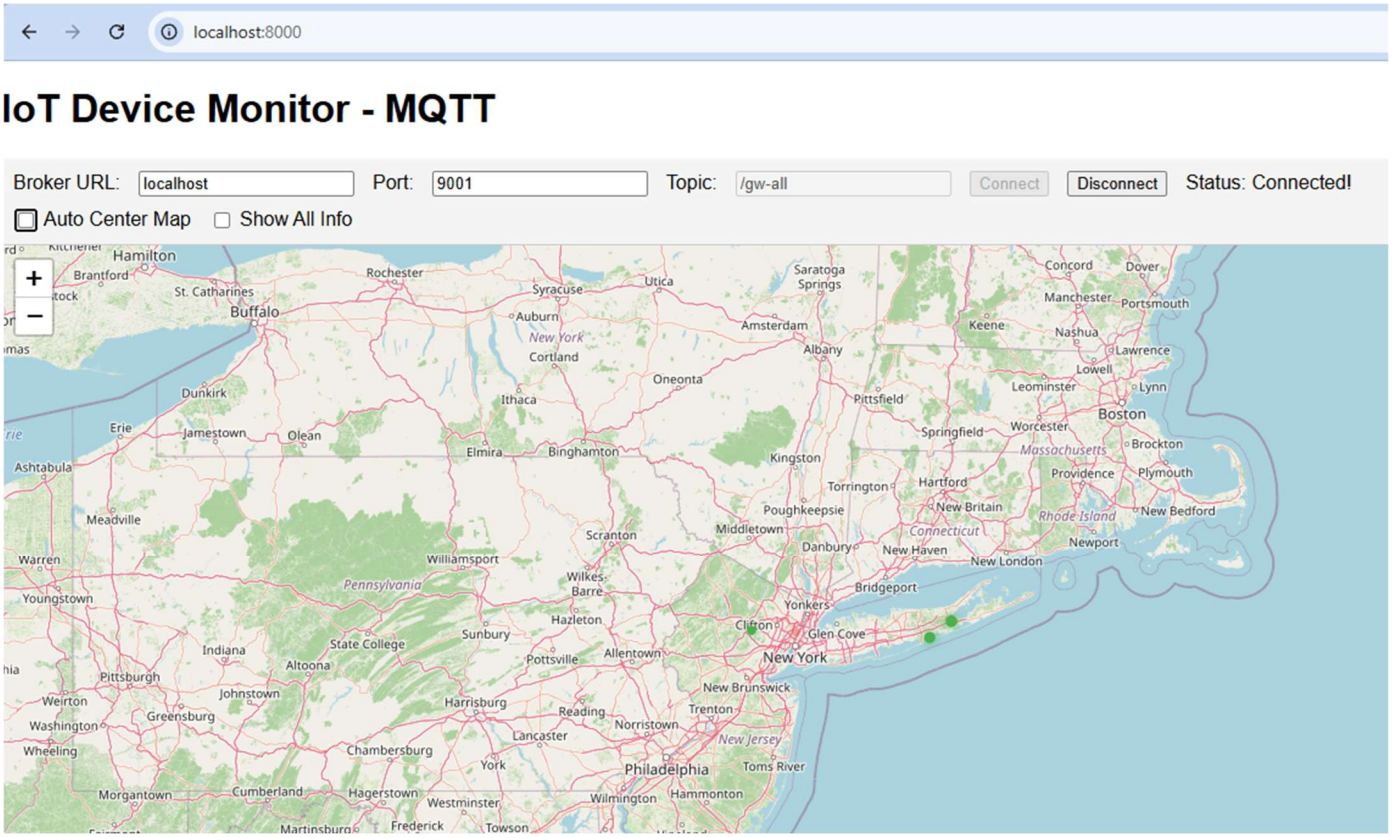


Figura - Monitor MQTT - Devices.

12. Eventos manuais com iot-client

Para os primeiros testes, deixe scenarios/device-model.json rodando na porta 5001.

12.1 Estado inicial

```
1 – Inicia Device:
uv run iot-sim scenarios/device-model.json

2 – Consultando o Dispositivo
uv run iot-client coap://127.0.0.1:5001/device/data
```

Anote status, bateria, temperatura, pressão e coordenadas. Esse é o estado de referência antes do evento.

12.2 Evento transient

```
uv run iot-client coap://127.0.0.1:5001/device/data scenarios/event-transient.json
```

Esse arquivo é propositalmente forte: drop_percentage chega a 80% e o delay passa para 5-12 s. A sequência aproximada é 20 s de entrada + 40 s ativo + 30 s de retorno. Durante o evento, alguns GETs podem demorar ou não responder por causa da perda simulada.

Para acompanhar, repita em outro terminal:

```
watch -n 2 'uv run iot-client coap://127.0.0.1:5001/device/data'
```

12.3 Evento permanente

```
uv run iot-client coap://127.0.0.1:5001/device/data scenarios/event-permanent.json
```

O arquivo usa transition_duration_s=120. Depois da transição, o novo estado permanece. Para voltar rapidamente ao estado inicial, encerre o dispositivo com Ctrl+C e inicie novamente scenarios/device-model.json.

Para acompanhar, repita no terminal:

```
watch -n 2 'uv run iot-client coap://127.0.0.1:5001/device/data'
```

12.4 Evento de movimento - Uberlândia

Para testar o deslocamento de um dispositivo até Uberlândia/MG, crie um arquivo de evento dentro da pasta scenarios do projeto.

```
vi scenarios/event-moving-udi.json
{
  "event_name": "Movimento para Uberlandia",
  "coordinate": {
    "latitude": -18.9186,
    "longitude": -48.2772
  },
  "transition_duration_s": 100,
  "event_type": "transient"
}
```

```
1 – Inicia Device:
uv run iot-sim scenarios/device-model.json

2 – Publica no Mosquitto:
uv run iot-gw -i 1000 -d scenarios/devices-moving.json -b localhost:1883 -t "/gw/moving/"

3 – Aplica Evento Movimento:
uv run iot-client coap://127.0.0.1:5001/device/data scenarios/event-moving-udi.json
```

Acompanhar via terminal – Esse comando será executado a cada 2 segundos

watch -n 2 'uv run iot-client coap://127.0.0.1:5001/device/data'

13. Event Manager (ev-man)

13.1 O que ele faz

- Lê um JSON que é uma lista de eventos.
- Ordena os itens por time_ms.
- Cria uma tarefa para cada item.
- Espera o tempo relativo ao início do programa.
- Envia CoAP POST para a URI indicada em device.

13.2 Executar o schedule do repositório

Primeiro deixe os quatro dispositivos do cenário 2x2 ligados com seu gateway para o mosquitto, conforme comandos abaixo

1 – Passo Iniciando os quatros dispositivos:

uv run iot-sim-batch --config-dir scenarios/2x2 --log-level INFO

2 – Passo Iniciando gateway para as regiões

uv run iot-gw -i 2000 -d scenarios/devices-2x2.json -b localhost:1883 -t "/gw-all"

3 – Passo Iniciando eventos agendados com Ev-Man

```
cd ~/iot-sim-main
uv run ev-man scenarios/schedule-rendezvous-2x2.json
```

Dispositivo	time_ms	Iniciar após	transition_duration_s
5001	1000	1 s	20 s
5002	7000	7 s	30 s
6001	11000	11 s	20 s
6002	15000	15 s	15 s

Todos recebem um evento chamado Rendezvous. O schedule altera coordinate, drop_percentage, delay_profiles e battery_idle_discharge.

14. Criando schedules próprios com ev-man

Criando um arquivo ev-man para testar:

14.1 Um dispositivo aos 10 segundos

```
vi scenarios/schedule-aula-1.json
```

Cole este conteúdo:

```
[
  {
    "time_ms": 10000,
    "device": "coap://127.0.0.1:5001/device/data",
    "event": {
      "event_name": "Movimento para Washington DC",
      "event_type": "permanent",
      "coordinate": {
        "latitude": 38.9072,
        "longitude": -77.0369
      },
      "delay_profiles": [],
      "transition_duration_s": 120
    }
  }
]
```

Execute:

```
uv run ev-man scenarios/schedule-aula-1.json
```

Com o time_ms=10000 significa disparar aproximadamente 10 s depois que o ev-man inicia. delay_profiles: [] significa que esse evento não troca os perfis de atraso já ativos no dispositivo.

15. Prática 3 - 10 dispositivos por região

Use os arquivos gerados na Seção 8.3: scenarios/2x10/reg1 e scenarios/2x10/reg2. Existem duas formas úteis de iniciar os 10 dispositivos.

15.1 Opção A - run-iot-range: um processo por dispositivo

Região 1:

```
./scripts/run-iot-range.sh scenarios/2x10/reg1 1 1 10
```

Região 2, em outro terminal:

```
./scripts/run-iot-range.sh scenarios/2x10/reg2 2 1 10
```

GW1 - Região 1:

```
uv run iot-gw -i 2000 -d scenarios/devices-01-10.json -b localhost:1883
-t "/gw/001/"
```

GW2 - Região 2:

```
uv run iot-gw -i 2000 -d scenarios/devices-02-10.json -b localhost:1883
-t "/gw/002/"
```

15.2 Opção B - iot-sim-batch: 10 dispositivos em um único processo

O comando do projeto chama-se iot-sim-batch e a opção --config-dir aponta para a pasta que contém os DeviceConfig.

Região 1:

```
uv run iot-sim-batch --config-dir scenarios/2x10/reg1 --log-level INFO
```

Região 2, em outro terminal:

```
uv run iot-sim-batch --config-dir scenarios/2x10/reg2 --log-level INFO
```

Cada comando acima mantém os 10 dispositivos daquela região dentro de um único processo Python, mas cada dispositivo continua ouvindo na sua própria porta UDP.

GW1 - Região 1:

```
uv run iot-gw -i 2000 -d scenarios/devices-01-10.json -b localhost:1883
-t "/gw/001/"
```

GW2 - Região 2:

```
uv run iot-gw -i 2000 -d scenarios/devices-02-10.json -b localhost:1883
-t "/gw/002/"
```

Importante: O --config-dir deve conter somente arquivos DeviceConfig. Por isso a Seção 9.3 moveu os arquivos com a lista dos devices para fora das pastas reg1/reg2.

15.3 Qual opção usar?

Forma	Processos	Quando usar
run-iot-range.sh	10 processos para 10 dispositivos	Bom para entender o modelo tradicional e controlar uma faixa.
iot-sim-batch	1 processo para 10 dispositivos	Bom para executar vários dispositivos com menos processos e observar tudo no mesmo servidor batch.

16. Execução em máquinas diferentes - o que muda

Primeiro valide tudo localmente. Depois, em um cenário distribuído, somente os endereços mudam:

Onde	Alteração
DeviceConfig	server_host deve usar o IP/interface da máquina que executa o dispositivo.
devices-XX.json	Cada URI deve apontar para o IP real da máquina dos dispositivos.
iot-gw	Em -b, use IP_DO_BROKER:1883.
Mapa	Broker URL deve ser o IP/hostname do broker visto pelo navegador; porta 9001.
ev-man	device deve conter uma URI CoAP alcançável pela máquina onde o ev-man roda.

Exemplo de gateway publicando em um broker na máquina 192.168.2.3:

```
uv run iot-gw -i 2000 -d scenarios/devices-01-10.json -b
192.168.2.3:1883 -t "/gw/001/"
```

Exemplo de destino remoto dentro de um schedule:

```
"device": "coap://192.168.2.20:5001/device/data"
```

Importante: Em máquinas diferentes, libere UDP nas portas CoAP dos dispositivos e TCP 1883/9001 no broker.